

THE ULTIMATE **RAG** **CHEAT SHEET**

Everything you need to build a
Production Ready Pipeline.

Ingestion → Retrieval → Evaluation



AI Coach John

Follow



PROITBRIDGE
DRIVE FOR BETTER FUTURE

↻ Repost

Architecture Pipeline

PHASE 1: INGESTION

Data Sources



Ingestion



Extraction



Cleaning



Chunking



Embeddings



Vector DB

PHASE 2: GENERATION

Vector DB



Retrieval



Re-ranking



Context Building



LLM Generation



Evaluation + Monitoring

Important Note: Most RAG failures happen **before** the LLM.



Data Ingestion

WHAT IT IS

Getting knowledge into your pipeline. The source matters. Different sources need different extraction strategies.

COMMON SOURCES



PDFs



Websites



Notion/Confluence



Databases



CSV/Excel



Slack/Emails



</> APIs

EXAMPLES

PDF

PyPDF, PDFPlumber, PyMuPDF, OCR

WEBSITE

BeautifulSoup, Playwright, Scrapy, Selenium, Firecrawl

API

REST API, GraphQL, Webhooks

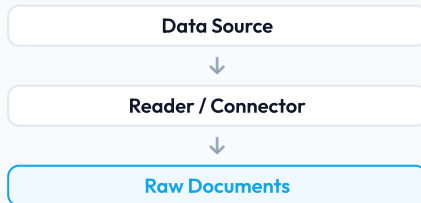
PRODUCTION WARNING

Bad ingestion = bad retrieval.
Garbage in. Garbage out.

HOW TO EXTRACT

SOURCE	EXTRACTION METHOD
PDF	OCR / PDF Parsers
Website	Web Scraping
APIs	API Connectors
DB	SQL Queries
Notion	SDK / API

FLOWCHART



Parsing & Cleaning

WHAT IT IS

Raw text is messy.
You need structured, usable text.

PRODUCTION TIP

Preserve: Source, Page Number,
Section Name.
Metadata becomes retrieval gold.

CLEANING TECHNIQUES

- ✓ Text normalization
- ✓ Deduplication
- ✓ Whitespace cleanup
- ✓ HTML stripping
- ✓ Language detection
- ✓ PII removal
- ✓ Metadata extraction

COMMON PROBLEMS

- ✗ Broken OCR
- ✗ HTML Noise
- ✗ Duplicate content
- ✗ Headers/Footers
- ✗ Encoding issues
- ✗ Tables lost

EXAMPLE

RAW

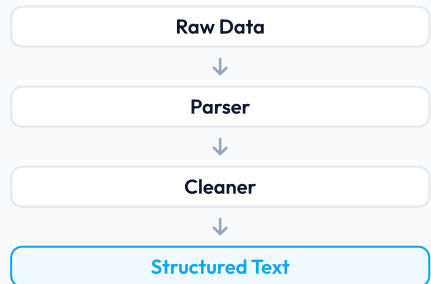
```
Page 4
Company Confidential
Revenue 2025 ...
```



CLEANED

```
Revenue 2025 ...
```

FLOWCHART



Chunking Cheat Sheet

? WHY CHUNKING EXISTS

LLMs cannot consume entire documents. We split intelligently.

📁 TECHNIQUES

- ✓ Fixed Chunking
- ✓ Semantic Chunking
- ✓ Hierarchical Chunking
- ✓ LLM Chunking
- ✓ Agentic Chunking

</> EXAMPLE

BEFORE

Entire 50 page report



AFTER

Chunk 1
Chunk 2
Chunk 3

🏠 QUICK DECISION TREE

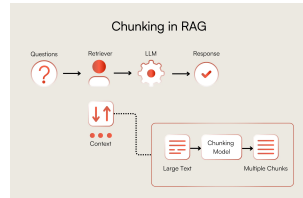
Clean docs?
→ **Fixed**

Multi-topic docs?
→ **Semantic**

Structured docs?
→ **Hierarchical**

Messy OCR?
→ **LLM**

Multiple formats?
→ **Agentic**



</> PYTHON EXAMPLE

```
from langchain.text_splitter import
RecursiveCharacterTextSplitter

splitter =
RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150
)
```



Embeddings

WHAT IT IS

Convert text into vectors.
Machines search vectors. Not words.

POPULAR MODELS

- OpenAI
- Cohere
- BGE
- E5
- Sentence Transformers
- Voyage

FLOW

"Python is a language"



[0.24, 0.93, 0.17, 0.61 ...]

Techniques in Embeddings in NLP

Types Of Word Embeddings

Frequency Based Embeddings

Bag Of Words
TF-IDF
Co-occurrence Matrices

Prediction Based Word Embedding

Word2Vec (CBOW, Skip-gram)
Glove
FastText

Contextualized Word Embeddings

ELMO
BERT
GPT

PRODUCTION TIP

Embedding model mismatch hurts recall.

Choose based on domain.

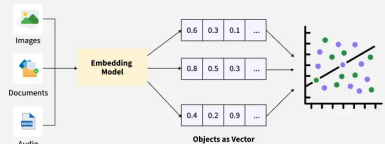
KEY CONCEPTS

- Dense Embeddings
- Sparse Embeddings
- Multimodal Embeddings
- Instruction Tuned

WHEN TO USE

Dense → semantic similarity
Sparse → keyword accuracy
Hybrid → production search

Multimodal Embedding



Vector DB

WHAT IT IS

Stores embeddings.
Enables similarity search.

POPULAR CHOICES

- ✓ Pinecone
- ✓ Qdrant
- ✓ Weaviate
- ✓ Milvus
- ✓ FAISS
- ✓ Chroma
- ✓ Redis

PRODUCTION TIP

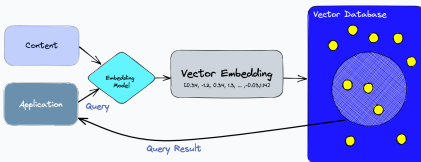
Metadata filtering dramatically improves retrieval.

RETRIEVAL TYPES

- ✓ KNN Search
- ✓ ANN Search
- ✓ Filtered Retrieval
- ✓ Metadata Search

STORED DATA

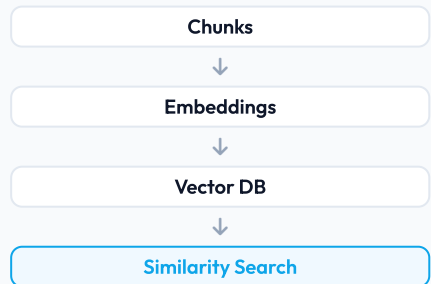
```
{
  "chunk": "JWT Authentication",
  "embedding": [...],
  "metadata": {
    "source": "API_DOC",
    "page": 12
  }
}
```



SCALING TIP

For >10M vectors, use **Quantization (PQ/SQ)** to reduce RAM usage by 4x-10x with minimal accuracy loss.

FLOWCHART



Metadata Filtering

WHAT IT IS

Search with constraints.
Not everything should be searched.

PRODUCTION BENEFIT

Higher precision.
Lower hallucinations.

EXAMPLE

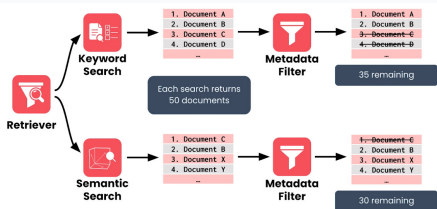
USER ASKS:

```
"2025 finance report"
```



FILTER APPLIED:

```
department = finance  
year = 2025
```



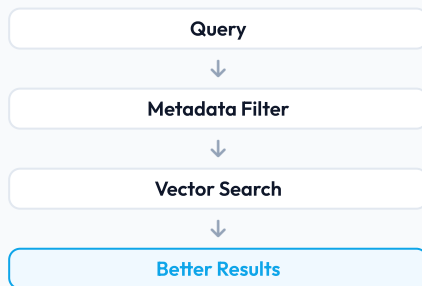
COMMON FILTERS

- Document Type
- Department
- Date
- Author
- Region
- Permission Level
- Language

PERFORMANCE TIP

Use **Pre-filtering** (filter before vector search) rather than Post-filtering to guarantee you don't miss top K results.

FLOWCHART



Hybrid Search

WHAT IT IS

Combine: **Semantic Search + Keyword Search**

FORMULA

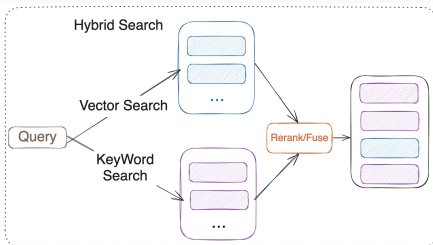
Dense Retrieval
+ Sparse Retrieval
= **Hybrid Search**

EXAMPLE

QUERY

"JWT token expiration policy"

- Dense finds meaning.
- Sparse catches exact keywords.



PRODUCTION USE CASES

- Enterprise Search
- Legal Docs
- Financial Data
- Compliance Systems

COMMON STACK

- ✓ BM25
- ✓ Dense Embeddings
- ✓ Fusion Ranking

ALPHA TUNING

- **alpha=1.0:** Pure Dense (Meanings)
- **alpha=0.0:** Pure Sparse (Keywords)
- **alpha=0.5:** Balanced (Best for RAG)

API EXAMPLE

```
vector_db.search(
    query="JWT errors",
    alpha=0.5 # 50% dense, 50%
    sparse
)
```



Query Transformation

WHAT IT IS

Rewrite user queries for better retrieval.

PRODUCTION BENEFIT

Massive recall improvement.

TECHNIQUES

- ✓ Query Expansion
- ✓ HyDE
- ✓ Step Back Prompting
- ✓ Sub-query Generation
- ✓ Multi Query Retrieval
- ✓ Self Query Retrieval

EXAMPLE

USER ASKS

Why is GPU memory full?



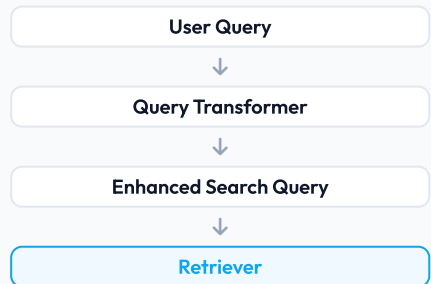
EXPANDED QUERY

CUDA OOM
GPU RAM
Memory Overflow

WHEN TO USE?

Use Query Transformation when user prompts are typically short, vague, or conversational (e.g., "how do I fix the bug?").

FLOWCHART



Re-Ranking

WHAT IT IS

Not every retrieved chunk deserves final context. Sort intelligently.

METHODS

✓ BM25 Ranking

✓ Cross Encoder

✓ ColBERT

✓ LLM Re-ranking

PIPELINE

Top 50 Retrieved



Cross Encoder

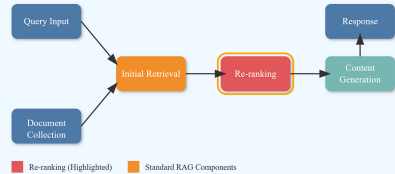


Top 5 Context Chunks

★ WHY IT MATTERS

- Better relevance.
- Less noisy context.
- Better final answers.

RAG Pipeline with Re-ranking



⚠ LATENCY TRADE-OFF

Cross-Encoders are **highly accurate but slow**. Only re-rank the top 25-50 results from your initial fast vector search.

</> PYTHON EXAMPLE

```

from sentence_transformers import
CrossEncoder

model = CrossEncoder('cross-
encoder/ms-marco')
scores = model.predict([
    (query, chunk_1),
    (query, chunk_2)
])
# Sort chunks by score descending
  
```



Context Construction

WHAT IT IS

Build the final prompt context.

PRODUCTION TIP

Too much context reduces accuracy.

TECHNIQUES

- ✓ Top-K Retrieval
- ✓ Sliding Context Window
- ✓ Compression
- ✓ Context Ordering
- ✓ Lost in Middle Fixes

EXAMPLE

```
Question
+ Chunk A
+ Chunk B
+ Chunk C
```



```
Final Prompt
```

"LOST IN THE MIDDLE"

LLMs focus on the **beginning and end** of a prompt. Place your highest-scoring chunks at the top and bottom of the context window.

PROMPT CONSTRUCTION

```
context_str = "\n".join([
    f"Source: {doc.text}"
    for doc in top_chunks
])

prompt = f"""
Answer based ONLY on this context:
{context_str}

User Question: {query}
"""
```



LLM Generation

WHAT IT IS

Generate grounded answers.

MODEL OPTIONS

- ✓ GPT
- ✓ Claude
- ✓ Gemini
- ✓ Llama
- ✓ Mistral
- ✓ DeepSeek
- ✓ Open Source

PROMPTING METHODS

- ✓ System Prompting
- ✓ Chain of Thought
- ✓ Citation Prompting
- ✓ Guardrails
- ✓ Structured Output

PRODUCTION GOAL

Grounded answers. Not creative hallucinations.

</> EXAMPLE PROMPT

```
Answer only using retrieved
context.
Return citations.
```

🛡️ HALLUCINATION DEFENSE

Instruct the LLM explicitly:
"If the context does not contain the answer, say 'I don't know'."

</> JSON OUTPUT EXAMPLE

```
{
  "answer": "RAG stops
hallucinations ...",
  "citations": [
    {"doc": "policy.pdf", "page":
4}
  ]
}
```



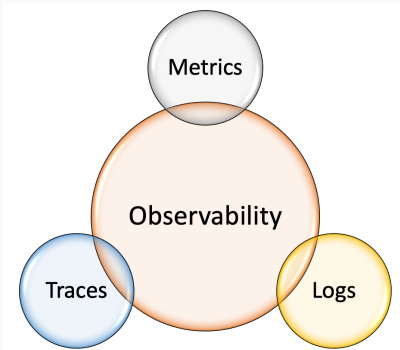
Observability

WHAT IT IS

Debugging RAG in production.

TRACK EVERYTHING

- ✓ Latency
- ✓ Retrieval Quality
- ✓ Token Usage
- ✓ Chunk Recall
- ✓ Failure Cases
- ✓ Hallucinations



TOOLS

- ✓ LangSmith
- ✓ Phoenix
- ✓ Helicone
- ✓ OpenTelemetry
- ✓ Weights & Biases

CRITICAL METRIC

Monitor your **Chunk Recall Rate**. If the right chunk is never retrieved, the LLM will always fail.

FLOWCHART



Evaluation

WHAT IT IS

Measure if your RAG actually works.

EVALUATION TYPES

- ✓ Retrieval Eval
- ✓ Answer Eval
- ✓ End-to-End Eval
- ✓ Human Eval

METRICS

- ✓ Recall@K
- ✓ Precision@K
- ✓ MRR
- ✓ Faithfulness
- ✓ Answer Relevance
- ✓ Latency
- ✓ Cost

⚖️ MEASUREMENT REALITY

Good Retrieval. Bad Answer.

OR

Bad Retrieval. Good Prompt.

Need separate measurement.

↻ CONTINUOUS EVAL

Run evaluation datasets (100+ Q&A pairs) on every PR to ensure prompt or embedding tweaks don't regress performance.

</> RAGAS EVAL FRAMEWORK

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy
)

results = evaluate(dataset,
    metrics=[
        faithfulness, answer_relevancy
    ])
```

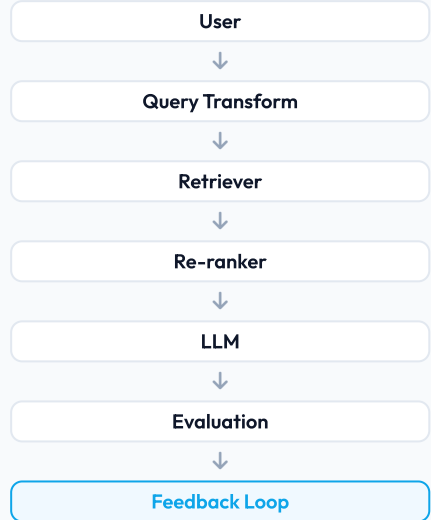


Production Extras

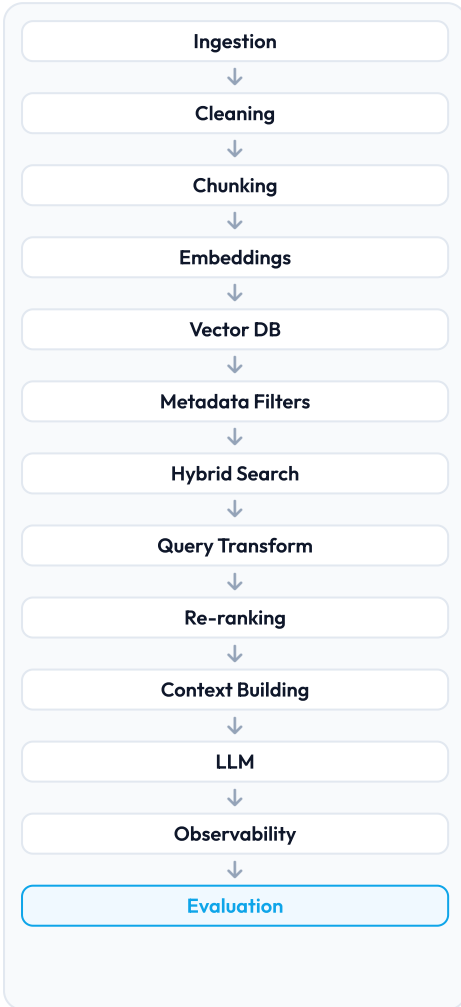
+ ADD THESE IN REAL SYSTEMS

- ✓ Caching
- ✓ Guardrails
- ✓ Access Control
- ✓ Streaming
- ✓ Feedback Loops
- ✓ Agentic Retrieval
- ✓ Multi-Hop Retrieval
- ✓ Knowledge Graph RAG
- ✓ Multimodal RAG
- ✓ Versioning

🧩 MINI ARCHITECTURE



Cheat Sheet **Summary**



FINAL TAKEAWAY:

**RAG is not just
Vector DB + GPT.**

**Production RAG is an
engineering system.**

